

Aula 2 — Playwright Avançado

"Você já sabe o que testar. Agora vamos automatizar."

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 01/10/2026

Recap da Aula 1

- ISO/IEC 25010:2023 (9 características), Pirâmide vs Trophy vs Honeycomb
- Teste exploratório: FEW HICCUPPS + Charter (SBTM)
- Web/mobile por dentro (render pipeline, lifecycle), acessibilidade, heurísticas de Nielsen
- Lab manual: app TestesQAMobile (13+ bugs) + exploratório no arngren.net

Atividade 1 entregue? (Módulos 1, 2, 3 do app + arngren.net — prazo era hoje)

| Hoje muda de fase: tudo que vocês testaram na unha, começamos a automatizar.

Agenda da aula (3h30)

- **90min** — Playwright avançado: arquitetura interna, locators, network mocking, visual regression (teoria + demo guiada)
- **15min** — Intervalo
- **90min** — Lab hands-on em sala: Playwright SPA (live coding com replicação pelos alunos)
- **15min** — Wrap-up

Objetivos M2

- **Compreender** arquitetura interna do Playwright (CDP, contexts)
- **Aplicar** locators recomendados (role, text, test-id)
- **Implementar** network mocking + HAR replay
- **Configurar** storageState para auth reuse
- **Aplicar** visual regression nativo (`toHaveScreenshot`)
- **Configurar** sharding 4-way em CI

O que é o Playwright?

Framework de automação de browser — controla Chromium, Firefox e WebKit de fora, como um robô clicando e digitando, mas via protocolo, não coordenadas de tela.

Origem: criado pela **Microsoft**, lançado janeiro/2020. Time = os mesmos engenheiros que criaram o **Puppeteer** no Google, saíram e foram pra Microsoft em 2019 — usaram a experiência pra corrigir as limitações do Puppeteer (só Chromium).

3 coisas que o Puppeteer nunca teve: suporte a Firefox e WebKit (não só Chrome) · API multi-página pensada pra SPA · auto-waiting nativo (sem `waitForSelector` espalhado pelo código).

Fonte: [Wikipedia, Playwright \(software\)](#)

Motores de renderização — Chromium, Gecko, WebKit

Motor de renderização (rendering engine) é o programa que lê HTML/CSS/JS e decide o que pintar na tela — o "cérebro" de dentro do navegador. Cada navegador roda em cima de 1 motor:

Chromium

engine Blink · Google

Chrome · Edge · Opera · Brave

Gecko

Mozilla

Firefox

WebKit

Apple

Safari · todo browser no iOS

Motores diferentes podem renderizar o MESMO HTML/CSS de jeito diferente — um bug visual que só aparece no Safari não aparece no Chrome. Por isso "cross-browser" é 1 dos pilares do Playwright: a mesma API testa os 3.

Instalando e gerando testes sozinho

```
# Cria projeto novo com tudo configurado (TS, config, exemplo)
npm init playwright@latest

# Baixa os browsers (Chromium, Firefox, WebKit) — ~300MB
npx playwright install
```

Codegen — grava sua navegação e gera o teste sozinho:

```
npx playwright codegen https://demo.playwright.dev/todomvc
```

Abre um browser real. Você clica, digita, navega normalmente — o Playwright **grava cada ação** e escreve o código do teste ao vivo numa janela ao lado (locators já na forma recomendada). Ótimo pra começar sem saber a API de cor.

| Rodem isso agora, ao vivo, em qualquer site — depois a gente edita o que ele gerou.

Extensão oficial do VS Code — Codegen sem terminal

Tudo que o `codegen` faz pelo terminal, a extensão **Playwright Test for VSCode** (Microsoft, Marketplace) faz direto no editor — clicando, sem decorar comando:

```
Extensions (Cmd/Ctrl+Shift+X) → busca "Playwright Test" → Install
```

Depois de instalada, aparece um ícone novo na sidebar esquerda: o **frasco de Erlenmeyer** () — é a aba **Testing**. É lá que mora tudo que vem a seguir.

Já vem instalada no ambiente de vocês. Se não aparecer o frasco, reinicie o VS Code.

Aba Testing — os botões que importam

Botão	O que faz	Grava teste?
▷ Record new	Abre um browser real, você navega, o código aparece ao vivo no editor	✓ sim, do zero
Record at cursor	Mesma ideia, mas continua um teste que já existe a partir de onde o cursor está	✓ sim, anexando
 Pick locator	Abre o browser só pra você apontar 1 elemento e copiar o seletor certo	✗ não — só o locator
► Run Test (com "Show browser" marcado)	Executa um teste já escrito, mostrando o browser em vez de rodar headless	✗ não — só executa

Diferença chave: Record new/at cursor **escrevem código pra você**. Pick locator só **devolve o seletor** — você cola ele onde quiser no teste.

UI Mode — `--ui`

```
npx playwright test --ui
```

Abre uma janela separada (não é bem "um browser") com:

- **Lista de todos os testes** à esquerda, roda um por um ou todos de uma vez
- **Timeline + trace** de cada passo — clica num passo e vê a tela exatamente naquele momento
- **Watch mode** — salvou o arquivo, já roda de novo sozinho

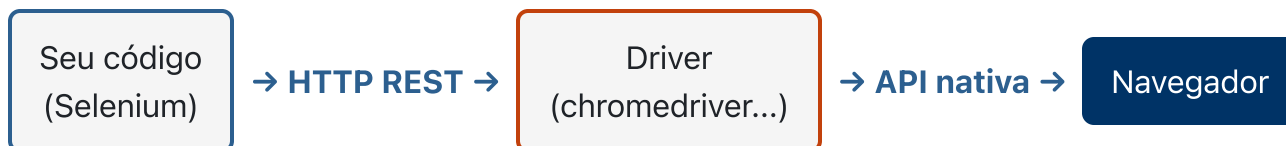
Pra debugar teste que falhou: UI Mode é mais rápido que ficar lendo stack trace no terminal.

Use bastante na Atividade/Lab.

O que é WebDriver?

WebDriver não é um produto — é um protocolo, um padrão W3C (recomendação oficial desde 2018) que define um jeito universal de um programa controlar um navegador: "abra esta URL", "clique neste elemento", "digite este texto". Nasceu do projeto **Selenium** (2004+) e virou o padrão que praticamente todo navegador passou a suportar.

Tem sempre um **driver** (processo separado, 1 por navegador: `chromedriver`, `geckodriver`, `safaridriver`) no meio, traduzindo:



Cada comando (clicar, digitar) = 1 chamada HTTP REST pro driver, que traduz pra API nativa daquele navegador específico.

Por que Playwright?

Em 2026, Playwright tomou espaço de Selenium e Cypress em times sérios.

Razões técnicas:

1. **Chrome DevTools Protocol** (não WebDriver) — controle granular
2. **Browser contexts isolados** — paralelismo limpo
3. **Auto-waiting nativo** — sem sleeps
4. **Locators recomendados** — role, text, test-id
5. **Trace viewer** — debug determinístico
6. **Cross-browser** — Chromium, Firefox, WebKit
7. **Visual regression nativo** — `toHaveScreenshot`

Arquitetura — CDP vs WebDriver

CDP (Chrome DevTools Protocol) é o protocolo nativo do Chromium — o MESMO que o DevTools usa quando você aperta F12. Playwright fala CDP **direto com o navegador**, sem driver no meio, por **WebSocket** (conexão única, persistente, bidirecional — não 1 request HTTP por comando).

E Firefox/WebKit, que não têm CDP nativo? Playwright usa **builds patchados** desses navegadores (por isso `playwright install` baixa binários próprios, não usa o Firefox/Safari já instalado na sua máquina) — patches que expõem um protocolo equivalente ao CDP, mantido pelo próprio time do Playwright.

CDP vs WebDriver — na prática



	WebDriver	CDP
Intermediário	driver externo, 1 por browser	nenhum (Chromium) / browser patchado
Conexão	HTTP, 1 req = 1 comando	WebSocket persistente
Acesso extra	só o que o padrão define	network, performance, runtime

Diferença prática: Playwright é tipicamente 2–4x mais rápido em mesma suíte.

Browser context — isolamento

Um **context** é uma sessão de navegador isolada — tipo abrir uma aba anônima nova — mas dentro do MESMO processo de browser já aberto. Não precisa lançar um Chromium inteiro pra cada sessão, só um context novo (leve, rápido).

```
const browser = await chromium.launch();

// Cada context é isolado: cookies, storage, history
const ctx1 = await browser.newContext({ locale: 'pt-BR' });
const ctx2 = await browser.newContext({ locale: 'en-US' });

const page1 = await ctx1.newPage();
const page2 = await ctx2.newPage();
```

| 1 browser, N contexts, M pages — paralelismo natural.

baseUrl — cuidado com o path

```
// playwright.config.ts
export default defineConfig({
  use: { baseUrl: 'http://localhost:4173' },
});
```

```
await page.goto('/login'); // ✓ vira http://localhost:4173/login
await page.goto('');       // ✓ vira o baseUrl exato
```

Pegadinha real: se o `baseUrl` já tem um path (`.../pasta/`), `goto('/algo')` **ignora** esse path e volta pra raiz do domínio — regra de resolução de URL, não bug do Playwright. Pra reaproveitar o path do `baseUrl` inteiro, use `goto('')`.

Achado ao vivo preparando o esquentado desta aula — se o teste "não acha nada" mesmo com locator certo, confira a URL de verdade com `console.log(page.url())`.

O que é um Locator?

Locator é um objeto do Playwright que aponta pra elemento(s) da página — mas não busca o elemento na hora que você *escreve* o código, busca na hora que você *usa* (`.click()` , `.fill()` ...), e repete a busca sozinho até achar (ou dar timeout).

Diferente de um `document.querySelector`, que busca uma vez só, no instante exato em que é chamado — se o elemento ainda não existe na tela naquele instante, falha na hora.

Locators recomendados

```
//  Bom — role + nome
page.getByRole('button', { name: 'Entrar' });
page.getByRole('link', { name: 'Sobre nós' });

//  Bom — texto visível
page.getByText('Welcome back');

//  Bom — test-id (estável)
page.getByTestId('submit-button');

//  Bom — placeholder, label
page.getByPlaceholder('Email');
page.getByLabel('Senha');

//  Ruim — frágil
page.locator('div.container > .card:nth-child(2) span.title');
```

O que é auto-waiting?

Frameworks mais antigos (Selenium cru) não esperam sozinhos: se o botão ainda não carregou quando o código tenta clicar, dá erro na hora ou clica errado.

A saída de quem não tem auto-waiting é `sleep(2)` fixo (lento, e ainda quebra se a rede ficar mais lenta que 2s) ou escrever um `WebDriverWait` explícito à mão pra CADA elemento. Playwright faz isso sozinho, pra toda ação.

Auto-waiting — sem sleeps

```
// Playwright aguarda elemento estar:  
// - Anexado ao DOM  
// - Visível  
// - Estável (não animando)  
// - Habilitado  
// - Receivável (não coberto)  
  
await page.getByRole('button', { name: 'Entrar' }).click();  
// Já aguardou tudo automaticamente  
  
// Web-first assertions também aguardam  
await expect(page.getByText('Welcome')).toBeVisible({ timeout: 5000 });
```

Se ainda precisa de `waitForTimeout`, está fazendo errado.

O que é flakiness?

Teste flaky: mesmo código, mesma branch, comportamento diferente entre runs — passa agora, falha daqui a pouco, sem ninguém ter mudado nada.

Por que é ruim: quando um teste falha "às vezes", o time para de confiar no vermelho — roda de novo até passar em vez de investigar ("deve ser flaky"). Aí um bug REAL também vira "deve ser flaky" e passa despercebido. Teste que não é confiável é pior que não ter teste: dá falsa sensação de segurança.

Flakiness — métricas

```
Flake rate = runs flaky / total runs
```

Targets:

- Time elite: < 1% flake rate
- Time saudável: < 5%
- Time problemático: > 10%

```
# Detectar flaky em CI  
npx playwright test --repeat-each=3  
# Testes que falharam em 1 ou 2 (não 0 nem 3) são flaky
```

Flakiness — mitigação

- ✓ **Auto-waiting** (Playwright nativo — slide anterior)
- ✓ **Locators recomendados** (role, text, test-id) > seletores frágeis
- ✓ **Isolamento entre testes** (storageState, browser context)
- ✓ **Retry inteligente** em CI (1–2 max, com investigação)
- ✗ **Sleep fixo** (`waitForTimeout`) — anti-pattern absoluto
- ✗ **Retry em massa** (`retries: 10`) — mascara bugs reais
- ✗ **Skip silencioso** — esconde problemas

Regra prática: cada flake tem causa raiz. "Rodei de novo e passou" não é debug, é procrastinação.

O que é mocking?

Mocking = interceptar uma chamada de rede ANTES dela sair de verdade, e devolver uma resposta fake que você controla — em vez de bater no backend real.

Serve pra testar casos difíceis de provocar de verdade (erro 500, timeout, API fora do ar) sem derrubar um servidor de propósito, e deixa o teste rápido e determinístico (não depende de rede/backend estarem de pé).

Network mocking — `route()`

```
test('handles API error gracefully', async ({ page }) => {  
  // Intercepta antes da request  
  await page.route('**/api/checkout', async (route) => {  
    await route.fulfill({  
      status: 500,  
      contentType: 'application/json',  
      body: JSON.stringify({ error: 'Internal Server Error' }),  
    });  
  });  
  
  await page.goto('/checkout');  
  await page.getByRole('button', { name: 'Pagar' }).click();  
  
  await expect(page.getByText('Erro. Tente novamente.')).toBeVisible();  
});
```

O que é HAR?

HAR (*HTTP Archive*) é um arquivo `.json` que registra TODAS as requisições e respostas de uma sessão de navegação — o mesmo que a aba Network do DevTools exporta em "Save all as HAR".

Em vez de mockar rota por rota com `route()` (slide anterior), você grava a sessão real 1 vez e manda o Playwright **reproduzir esse tráfego gravado inteiro** depois — útil quando o fluxo bate em dezenas de endpoints e mockar cada um à mão seria trabalho demais.

HAR replay

```
// Capturar  
const context = await browser.newContext({  
  recordHar: { path: 'session.har' },  
});  
  
// Replay  
await page.routeFromHAR('session.har', {  
  url: '**/api/**',  
  notFound: 'fallback',  
});
```

Útil para testar fluxos complexos sem depender de backend rodando.

storageState — auth reuse

storageState = JSON com cookies + localStorage salvos após login — carregar esse arquivo num teste novo pula a tela de login (já abre "logado"), em vez de repetir o fluxo em cada teste.

Sem reuso: 3s × 200 testes = 10min só de login.

```
// global-setup.ts — roda 1 vez
async function globalSetup() {
  const browser = await chromium.launch();
  const page = await browser.newPage();
  await page.goto('/login');
  await page.fill('[name=email]', 'user@email.com');
  await page.fill('[name=password]', 'secret');
  await page.click('button[type=submit]');
  await page.waitForURL('/dashboard');
  await page.context().storageState({ path: 'auth.json' });
  await browser.close();
}
```

storageState — plugando no config

Com o `auth.json` gerado, basta apontar o `playwright.config.ts` pra ele — toda suíte já começa logada:

```
// playwright.config.ts  
export default defineConfig({  
  globalSetup: require.resolve('./global-setup'),  
  use: { storageState: 'auth.json' },  
});
```

`globalSetup` roda 1 vez antes de TODA a suíte — não repete por teste nem por arquivo.

O que é visual regression?

Visual regression = comparar o screenshot de agora com um screenshot "bom conhecido" (baseline), pixel a pixel — pega mudança que nenhuma assertion de texto/DOM pegaria (cor errada, elemento desalinhado, fonte quebrada).

Como `toHaveScreenshot` funciona na prática:

1. **1ª vez que roda** (`--update-snapshots`): não tem baseline ainda → Playwright tira o screenshot e **salva** como `home.png` (essa é a baseline, vira parte do repo).
2. **Próximas vezes**: tira um screenshot novo e compara pixel a pixel contra o `home.png` salvo.
3. Diferença maior que `maxDiffPixels` → **teste falha**, e o relatório mostra os 3: baseline, atual, e o diff (destacado).

Visual regression nativo

```
test('home page visual', async ({ page }) => {  
  await page.goto('/');  
  await expect(page).toHaveScreenshot('home.png', {  
    maxDiffPixels: 100,  
  });  
});  
  
// Tirar/atualizar a baseline:  
// npx playwright test --update-snapshots
```

Para múltiplos viewports, configure projects no `playwright.config.ts`.

O que é um trace?

Trace = gravação completa de 1 execução de teste (screenshot a cada ação + DOM + network + console) — como a caixa-preta de um avião, mas pra teste.

Se o teste falhou no CI, você não precisa reproduzir local: abre o trace e vê exatamente o que aconteceu, passo a passo.

Trace viewer — debug determinístico

```
# playwright.config.ts: use: { trace: 'retain-on-failure' }  
npx playwright show-trace trace.zip
```

UI mostra: timeline de cada ação · DOM snapshots a cada step · network requests · console logs · source code do teste.

Game changer para debug. Sem precisar reproduzir local.

Component Testing

Component testing = montar e testar 1 componente de UI isolado (não a aplicação inteira — sem navegar rota, sem backend) dentro de um browser real. Fica entre unit test (sem browser, mas também sem CSS/layout real) e E2E (app inteiro, mais lento). Alternativa a Cypress CT.

```
// LoginForm.spec.tsx
import { test, expect } from '@playwright/experimental-ct-react';
import LoginForm from './LoginForm';

test('disables submit when email invalid', async ({ mount }) => {
  const component = await mount(<LoginForm />);
  await component.locator('[name=email]').fill('invalid');
  await expect(component.locator('button[type=submit]')).toBeDisabled();
});
```

Roda componente isolado em browser real, sem app inteiro. Tempo médio entre unit e E2E.

Page Object Model (POM)

Padrão clássico. Encapsular interação com página em uma classe — o teste não conhece seletores, só a intenção.

```
class LoginPage {  
  constructor(private page: Page) {}  
  emailInput = this.page.getByPlaceholder('Email');  
  submitButton = this.page.getByRole('button', { name: 'Entrar' });  
  async login(email: string, password: string) {  
    await this.emailInput.fill(email);  
    await this.page.getByPlaceholder('Senha').fill(password);  
    await this.submitButton.click();  
  }  
}  
  
test('login flow', async ({ page }) => {  
  await new LoginPage(page).login('user@email.com', 'secret');  
  await expect(page).toHaveURL('/dashboard');  
});
```

POM — por que (e quando não)

Por que: seletor mudou? Corrige em 1 lugar (a classe), não em 50 testes. Teste lê como intenção de negócio, não como manipulação de DOM.

Fixtures do Playwright são a evolução natural: injetam a Page Object pronta via `test.extend()`, sem `new LoginPage(page)` repetido em todo teste.

```
export const test = base.extend<{ loginPage: LoginPage }>({
  loginPage: async ({ page }, use) => {
    await use(new LoginPage(page));
  },
});
```

POM não é critério avaliado à parte na rubrica dos 20pts (os 6 critérios são storageState, mocking, visual, SW, offline, Lighthouse) — mas é a base do desafio "difícil" de hoje (próximo bloco). Fixtures é mais idiomático em Playwright 2026, mas os dois valem pra esse desafio.

Screenplay Pattern — pra saber que existe

Evolução do POM: modela **Actor, Ability, Task, Question** em vez de classes por página.

```
const user = Actor.named('Maria').whoCan(BrowseTheWeb.using(page));  
await user.attemptsTo(Login.withCredentials('user@email.com', 'secret'));  
await user.asks(TheTitle.ofPage()).should(equal('Dashboard'));
```

Mais verboso, mais expressivo — vale a pena em suítes grandes com muito reuso entre fluxos. **Não vamos aprofundar hoje** (POM já resolve 90% dos casos reais); se quiser se aprofundar: [Serenity/JS](#), [The Screenplay Pattern](#).

O que é sharding?

Sharding = dividir a suíte de testes em N pedaços (*shards*) que rodam em **paralelo**, cada um numa máquina/job diferente.

400 testes em 4 shards = ~100 testes por máquina ao mesmo tempo → tempo total de CI cai ~4x.

Sharding 4-way em CI

```
# .github/workflows/playwright.yml
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        shard: [1/4, 2/4, 3/4, 4/4]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - run: npm ci && npx playwright install --with-deps
      - run: npx playwright test --shard=${{ matrix.shard }}
      - uses: actions/upload-artifact@v4
        with: { name: blob-report-${{ matrix.shard }}, path: blob-report }
```

Cada shard sobe seu próprio `blob-report` — agregação no próximo slide.

Blob reports — agregação

Cada shard gera seu próprio relatório (`blob-report`) — sem agregação você teria 4 relatórios separados pra abrir. Este job final baixa os 4 e junta num relatório HTML único.

```
merge-reports:
  needs: test
  runs-on: ubuntu-latest
  steps:
    - uses: actions/download-artifact@v4
    - run: npx playwright merge-reports --reporter html ./blob-report
    - uses: actions/upload-artifact@v4
      with: { name: html-report, path: playwright-report }
```

Resultado: 1 relatório consolidado de 4 shards paralelos.

`test.fail()` — documentando um bug conhecido

Assertion falhando não é sempre "meu teste está quebrado" — às vezes é "o produto tem um bug e eu quero rastrear isso".

```
test('bug conhecido: campo aceita e-mail inválido', async ({ page }) => {  
  test.fail(); // "eu SEI que essa asserção falha"  
  await expect(page.getByText('E-mail inválido')).toBeVisible();  
});
```

- Assertion **falha** (bug ainda existe) → Playwright reporta como "**expected failure**" → CI fica **verde**.
- Assertion **passa** (bug foi corrigido!) → Playwright reporta como **falha inesperada** → CI fica **vermelho** — alerta real de que algo mudou.

Isso vira um "ticket vivo": não precisa de Jira pra rastrear que um bug existe, o próprio teste documenta e avisa quando resolver.

Setup — antes de começar o lab

Node.js 22 instalado (`node -v`) — a suíte/CI do exercício é fixada nessa versão, versão diferente pode gerar comportamento diferente do que o CI vê.

```
node -v      # precisa aparecer v22.x
npm -v
```

Conta no GitHub (ativa, com acesso de leitura ao repo da disciplina) — a entrega é via **fork + Pull Request**, não upload de arquivo.

VS Code com a extensão **Playwright Test for VSCode** instalada (ver slides anteriores) — já vem no ambiente de vocês.

Fluxo de fork → clone → branch → PR é o deck `intro-fluxo-pr-github` (padrão de abertura da Aula 2). Se ainda não rodou, roda antes do lab — aqui é só aplicar.

Lab — trilha de dificuldade (90min em sala)

● Fácil — sem arquitetura (40min)

1. Fork + clone + `npm install` → `npm run test:e2e` roda "vazio" (10min)
2. Spec **02 — network mocking**: completar os TODOs (30min)

● Médio — já usa arquitetura pronta (30min)

3. Spec **03 — visual regression**: baseline em 3 viewports (20min)
4. `storageState / auth.setup.ts` (fixture já dada) — revisar juntos, não escreve do zero (10min)

Specs 04 (SW lifecycle) e 05 (offline) ficam pra Aula 3 — dependem de conteúdo que ainda não vimos. Rubrica (6 critérios, 20pts) no fim do bloco.

● Desafio difícil — em casa, criativo, CI verde

Terminou fácil + médio e quer ir além: desenhe e implemente, **sem scaffold comentado**, 1 cenário de teste pro Discover (/ , dado real do TMDb mockado no teste) cobrindo **paginação + abrir detalhe de um filme de página 2+ + "Ver comentários"** — organizado com **Page Object** (ex.: `DiscoverPage`, não funções soltas).

- CI precisa ficar **verde no fork** (mesmo workflow que já roda os specs 01-06).
- Vocês desenham o cenário do zero — criatividade conta.
- **Bônus — não entra nos 20pts da rubrica** (os 6 critérios abaixo continuam os únicos avaliados), mas vale destaque/feedback individual.

Dica: `06-discover-tmdb.spec.ts` já mostra o padrão de mockar `api.themoviedb.org` — usem de referência, não de template pra copiar.

Pitfalls comuns

- ✗ `waitForTimeout` em vez de auto-waiting
- ✗ Locators frágeis dependentes de hierarquia DOM
- ✗ Mock de tudo — testes desacoplam de realidade
- ✗ `retries: 3` sem investigar root cause de flake
- ✗ CI sem sharding — feedback de horas em suítes grandes
- ✗ Sem trace habilitado — debug demanda reproduzir local

Lab Web + PWA — 20 pts, Aula 2 + Aula 3

Entrega: fork + PR. App **CineFav Web** já pronto — vocês só escrevem os testes.

#	Critério	Pontos	Quando
1	Auth state reuse (<code>storageState</code>)	3	Hoje
2	Network mocking (<code>route()</code>)	3	Hoje
3	Visual regression, 3 viewports	4	Hoje
4	SW lifecycle testado	3	Aula 3
5	Offline mode (<code>setOffline</code>)	3	Aula 3
6	Lighthouse CI, 3 budgets	4	Aula 3

Eliminatório: suíte passa 100% em 3 runs consecutivos. Enunciado: `exercicios/01-lab-web-pwa-playwright/enunciado.md`.

Leituras obrigatórias

- [Playwright Docs — Best Practices](#) (oficial)
- Winteringham, M. — *Testing Web APIs* (Manning), cap 5–7
- Axelrod, A. — *Complete Guide to Test Automation* (Apress), cap 8–10
- [Google Testing Blog — Just Say No to More End-to-End Tests](#)
- [Serenity/JS — The Screenplay Pattern](#) (opcional, ver slide de Screenplay)

Próximo módulo (M3)

Testes de SPA & PWA

- Hydration testing
- Service Worker testing
- Offline mode tests
- Background Sync verification
- Lighthouse CI com budgets

Continuação do mesmo Lab Web + PWA (critérios 4-6) — não é um exercício novo.

Bons estudos!

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith

Office hours: quintas 19h